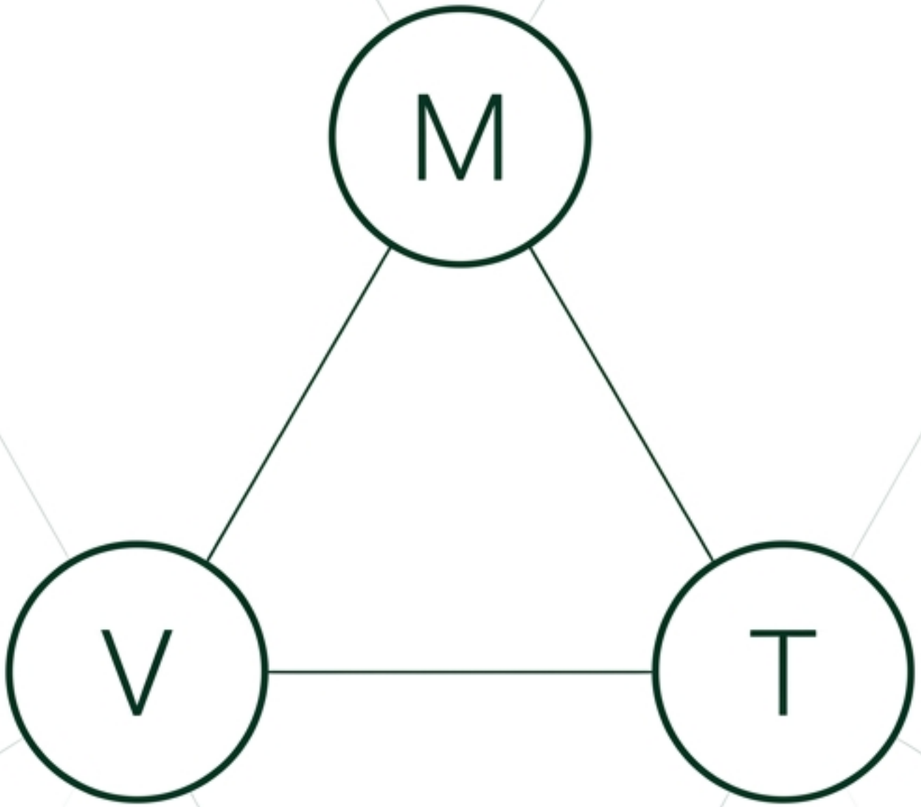


Arquitectura MVT en Django

Una inmersión profunda en Modelos, Vistas y Plantillas.

El framework web para perfeccionistas con fechas límite.

Gemini Notebook

¿Qué es Django?

Un framework web de alto nivel escrito en Python.



Rápido.

Fomenta el desarrollo ágil y el diseño pragmático para cumplir plazos.



Completo.

Filosofía 'Baterías incluidas'; todo lo necesario funciona en conjunto desde el primer día.



Escalable.

Arquitectura de componentes independientes (shared-nothing) que permite escalar a cualquier nivel.

Gemini Notebook

El Concepto MVT

M

Model (Capa de Datos).

Define la estructura y maneja la interacción con la base de datos.

V

View (Capa Lógica).

El cerebro de la aplicación; procesa la petición y orquesta el flujo de datos.

T

Template (Capa de Presentación).

Renderiza el HTML final que verá el usuario.

Gemini Notebook

El Modelo Mental: El Restaurante



Navegador web.

Hace el pedido (Petición HTTP).

Vista (View).

Recibe el pedido, va a la cocina, solicita los ingredientes y lleva el plato final.

Modelo (Model).

La despensa estructurada de donde se extraen los datos puros.

Plantilla (Template).

La presentación visual y estructurada del platillo final para el cliente.

Gemini Notebook

El Ciclo de Vida de una Petición

1. Usuario solicita una URL.

2. El enrutador (URL Mapper) dirige la petición a la Vista.

3. La Vista consulta al Modelo.

4. El Modelo extrae datos de la Base de Datos.

5. La Vista inyecta los datos en la Plantilla.

6. La Plantilla genera el documento HTML.

7. Se devuelve la respuesta HTTP al Usuario.



Gemini Notebook

Desmitificando el Acrónimo: MVT vs MVC

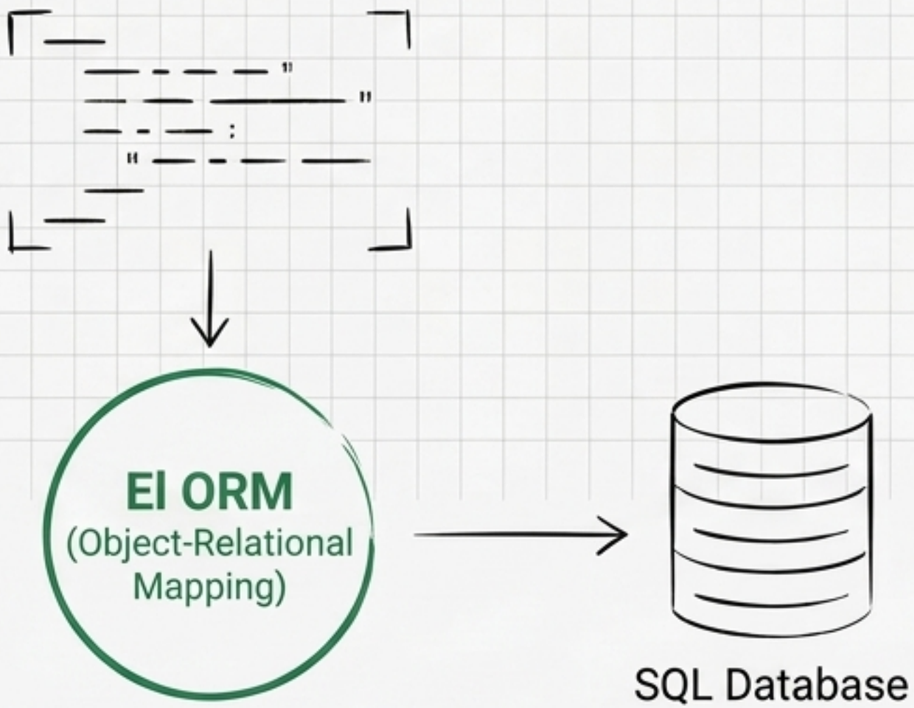
Concepto	Patrón MVC Tradicional	Patrón MVT (Django)
Manejo de Datos	Model	Model
Lógica del Controlador	Controller	View
Presentación Visual	View	Template

> ¿Dónde está el Controlador en Django? El framework en sí mismo (específicamente el sistema de URLs) actúa como el Controlador, delegando la lógica de negocio a las Vistas.

Gemini Notebook

M - El Modelo (La Capa de Datos)

La única fuente de verdad sobre tus datos. Contiene los campos y comportamientos esenciales de la información que estás almacenando.



El puente mágico que permite escribir código orientado a objetos (Python) sin necesidad de escribir sentencias SQL puras.

Gemini Notebook

Anatomía de un Modelo

CODE

```
class Tarea(models.Model):
    titulo = models.CharField(max_length=100)
```

DATABASE STRUCTURE



Gemini Notebook

El Poder del ORM y las Migraciones

Los modelos de Django abstraen por completo el DDL (Data Definition Language).

`models.py`

`python manage.py makemigrations`

Archivos de Migración
Inspecciona el modelo y crea un archivo de instrucciones de cambios.

`python manage.py migrate`

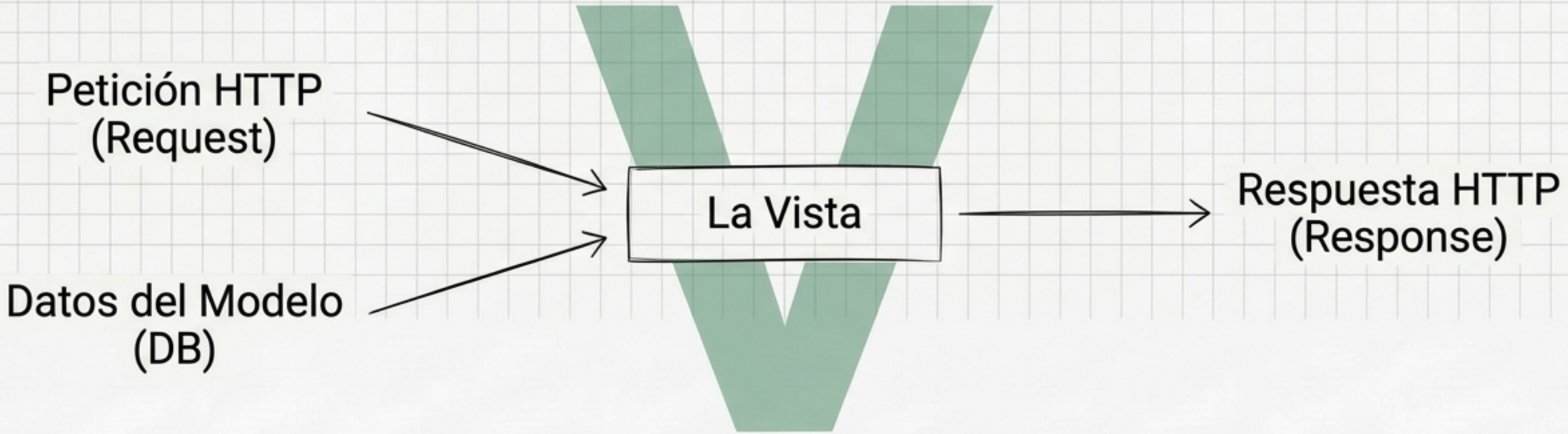


Base de Datos SQL
Traduce esas instrucciones a SQL y las ejecuta de forma segura.

Gemini Notebook

V - La Vista (La Capa Lógica)

El intermediario. Su función exclusiva es recibir una solicitud HTTP (request), aplicar la lógica de negocio necesaria, y retornar una respuesta HTTP.



- ✓ 1. Validar permisos y datos.
- ✓ 2. Consultar al Modelo.
- ✓ 3. Pasar el contexto a la Plantilla.

Gemini Notebook

Paradigmas de Vistas: Funciones vs Clases

Vistas Basadas en Funciones (FBV)

+ Simples y directas de leer.
+ Flujo de ejecución explícito.
+ Ideales para lógica altamente personalizada.

Vistas Basadas en Clases (CBV)

+ Fomentan la reutilización de código (DRY).
+ Heredan comportamientos (ej. ListView, CreateView).
+ Ideales para operaciones CRUD estándar.

Gemini Notebook

Anatomía de una Vista

```
def mi_vista(request):

    datos = Modelo.objects.all()

    contexto = {'items': datos}

    return render(request,
    'plantilla.html', contexto)
```

Entrada: Recibe el objeto HttpRequest.

Lógica: Interacción con el ORM para obtener datos.

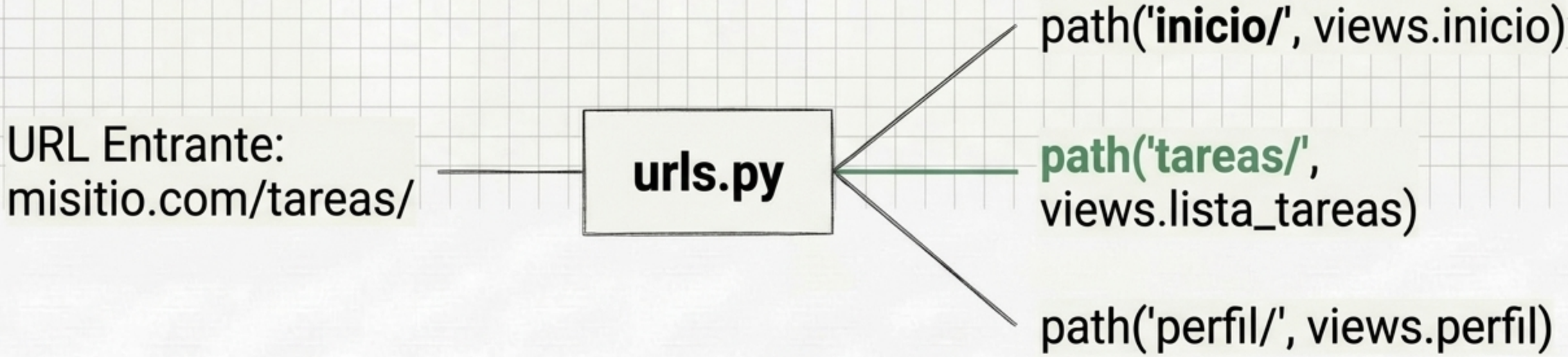
Empaquetado: Prepara los datos en un diccionario.

Salida: Retorna el HttpResponse renderizado.

Gemini Notebook

El Eslabón Perdido: Enrutamiento (URLs)

El **URL Mapper** (**urls.py**) intercepta la petición HTTP y busca coincidencias de patrones (paths) para ejecutar la Vista correcta.

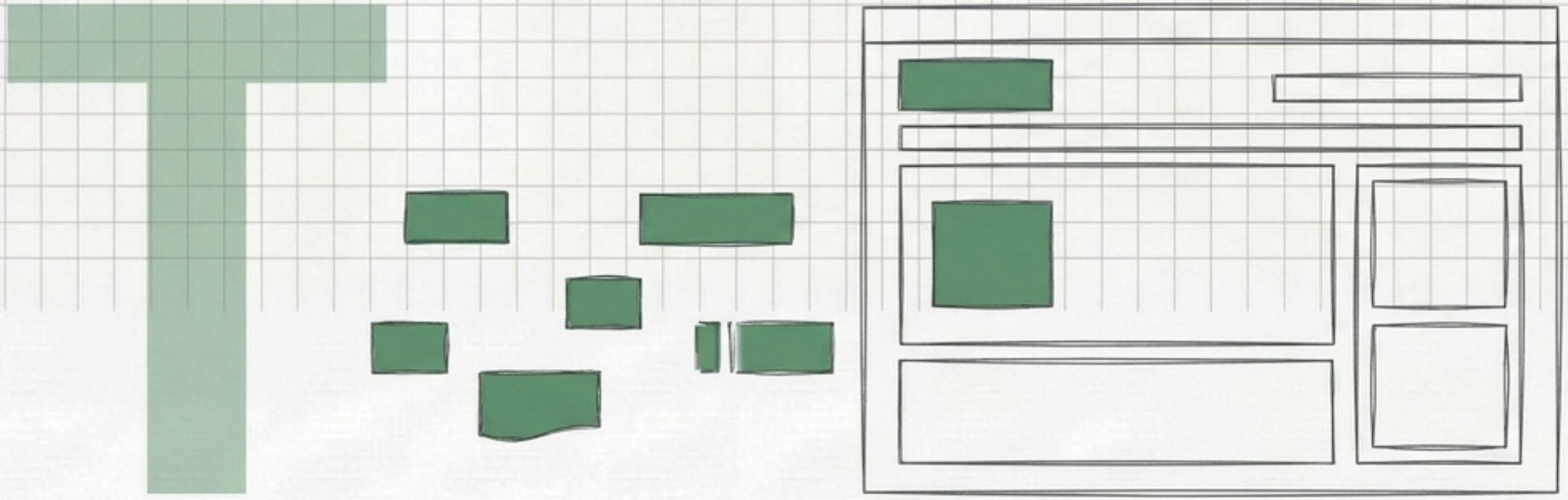


Redirige de forma elegante y permite capturar variables directamente desde la URL (ej. **<int:id>**).

Gemini Notebook

T - La Plantilla (Capa de Presentación)

Responsable de generar el HTML final. Su objetivo principal es separar estrictamente el diseño visual de la lógica de Python.



Mezcla HTML estático con el Lenguaje de Plantillas de Django (DTL) para inyectar contenido dinámico **de forma segura**.

Gemini Notebook

Sintaxis del Lenguaje de Plantillas

Variables {{ }}

Inyectan datos directamente desde el contexto de la Vista.

```
{{ nombre_usuario }}
```

Imprime el valor de la variable en el HTML.

Etiquetas (Tags) {% %}

Proporcionan lógica de control (bucles, condicionales) en la capa de presentación.

```
{% for item in items %}
...
{% endfor %}
```

Ejecuta un bucle sin usar Python puro.

Gemini Notebook

Anatomía de una Plantilla

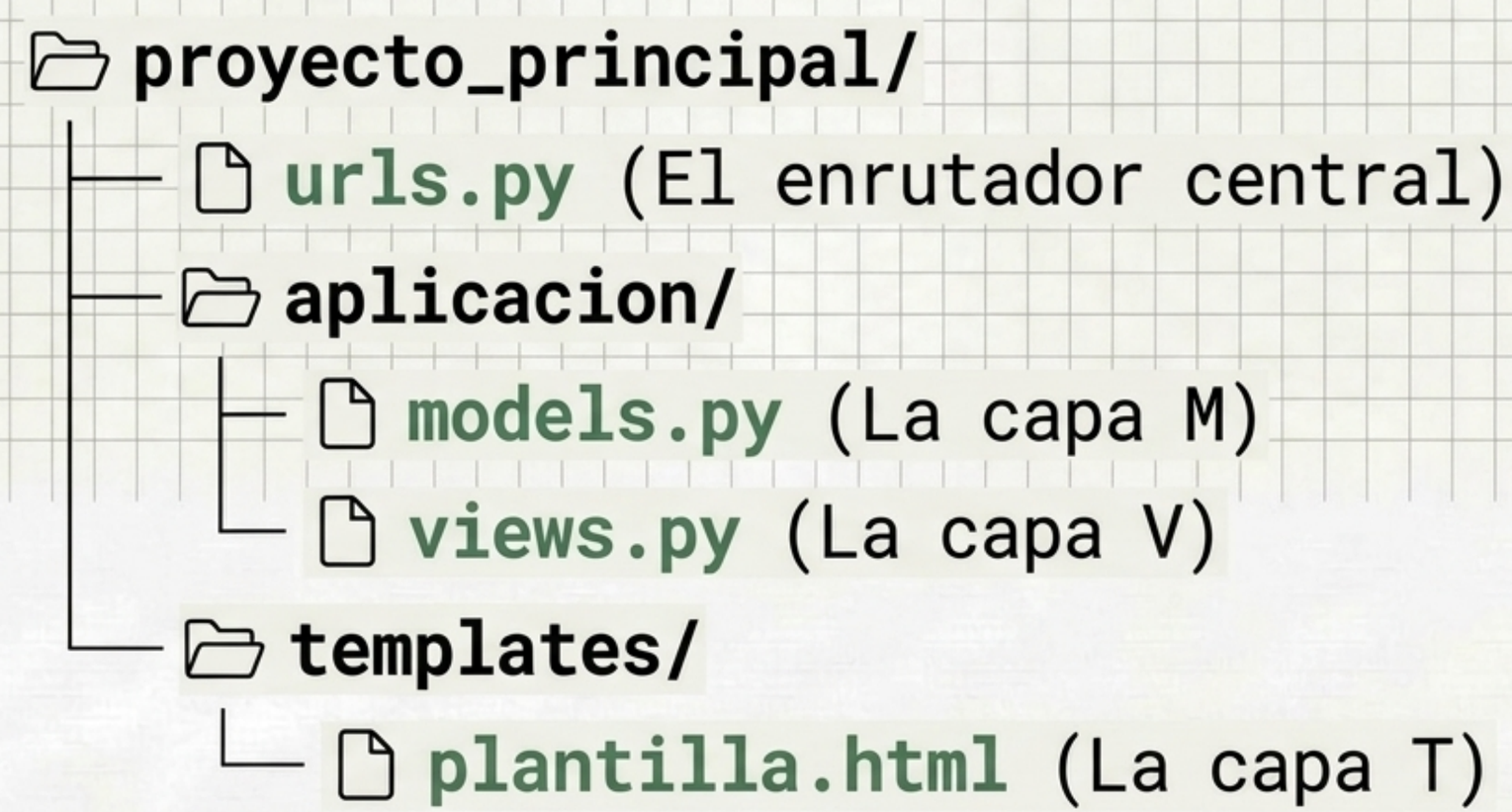
```
<ul>
  {% for tarea in tareas %}
    <li>{{ tarea.titulo }}</li>
  {% endfor %}
</ul>
```

Demuestra cómo la vista alimenta a la plantilla, iterando sobre una lista de objetos y extrayendo sus atributos sin ejecutar código Python real en el documento HTML.

Gemini Notebook

Arquitectura de la Información

Dónde viven estos conceptos en el ecosistema real de un proyecto Django.



Gemini Notebook

Caso Práctico: Lista de Tareas (1/3)

Paso 1: Definir el Modelo.

```
class Task(models.Model):
    title = models.CharField(max_length=100)
    completed = models.BooleanField(default=False)
```

Creamos la estructura en la base de datos para almacenar nuestras tareas.

Gemini Notebook

Caso Práctico: Lista de Tareas (2/3)

Paso 2: Configurar la Vista y la URL.

```
def task_list(request):
    tasks = Task.objects.all()
    return render(request, 'list.html', {'tasks': tasks})

path('tasks/', views.task_list)
```

Extraemos las tareas del Modelo y las preparamos para la Plantilla.

Gemini Notebook

Caso Práctico: Lista de Tareas (3/3)

Paso 3: Renderizar la Plantilla.

Plantilla Django (list.html)

```
<h1>Mis Tareas</h1>
<ul>
    {% for task in tasks %}
        <li>{{ task.title }}</li>
    {% endfor %}
</ul>
```

Salida Renderizada



El HTML final que el usuario recibe en su navegador.

Gemini Notebook

Síntesis: ¿Por qué MVT?



1. Desacoplamiento (Separation of Concerns).
Base de datos, lógica de negocio y diseño visual trabajan juntos sin pisarse. Modifica el diseño sin tocar la base de datos.

2. Eficiencia (DRY).
Don't Repeat Yourself. Código excepcionalmente limpio y fácil de mantener.

3. Desarrollo Rápido.
Permite a diseñadores frontend y desarrolladores backend trabajar en paralelo de forma segura.

La arquitectura perfecta para proyectos escalables y plazos ajustados.

Gemini Notebook